

Manipulação de Dados com SQL

Do modelo físico ao CRUD com PostgreSQL e Supabase



Prof. Dr. João Choma

Universidade Estadual de Maringá - DIN/CTC

github.com/JoaoChoma

Banco de Dados I ▪ SQL DML e consultas



Próxima etapa: os dados

Até este ponto trabalhamos principalmente com:

ESTRUTURA

Agora passaremos a trabalhar com:

DADOS

A próxima pergunta será:

Como inserir registros nas tabelas que acabamos de criar?

O primeiro comando será:

INSERT



Quando começarmos a manipular registros, teremos:

```
CREATE → INSERT  
READ   → SELECT  
UPDATE → UPDATE  
DELETE → DELETE
```

Esse é o CRUD aplicado aos dados.

Mas é importante não confundir:

CRUD dos dados

com:

DDL da estrutura



Já percorremos:

```
DER / MER
  ↓
MODELO RELACIONAL
  ↓
MODELO FÍSICO
  ↓
CREATE TABLE
  ↓
RESTRIÇÕES
  ↓
RELACIONAMENTOS
```

Agora:

```
TABELAS CRIADAS
  ↓
DADOS
  ↓
CRUD
```



Nesta disciplina, os comandos serão executados no:

SUPABASE



PostgreSQL

Portanto, os exemplos desta aula utilizam sintaxe PostgreSQL compatível com o **SQL Editor do Supabase**.



Vamos continuar com as tabelas:

CLIENTE

id_cliente (PK)

nome

email

ativo

criado_em

PEDIDO

id_pedido (PK)

data

valor

id_cliente (FK)

Relacionamento:

CLIENTE 1 ----- N PEDIDO



```
CREATE TABLE cliente (  
  id_cliente BIGINT GENERATED BY DEFAULT AS IDENTITY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT pk_cliente  
    PRIMARY KEY (id_cliente),  
  
  CONSTRAINT uk_cliente_email  
    UNIQUE (email)  
);
```



```
CREATE TABLE pedido (  
  id_pedido BIGINT GENERATED BY DEFAULT AS IDENTITY,  
  data TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  valor NUMERIC(10,2) NOT NULL,  
  id_cliente BIGINT NOT NULL,  
  
  CONSTRAINT pk_pedido  
    PRIMARY KEY (id_pedido),  
  
  CONSTRAINT ck_pedido_valor  
    CHECK (valor >= 0),  
  
  CONSTRAINT fk_pedido_cliente  
    FOREIGN KEY (id_cliente)  
    REFERENCES cliente(id_cliente)  
    ON DELETE RESTRICT  
);
```



Até agora fizemos:

```
CREATE TABLE ...  
ALTER TABLE ...  
DROP TABLE ...
```

Esses comandos trabalham com:

ESTRUTURA

Agora trabalharemos com:

```
INSERT  
SELECT  
UPDATE  
DELETE
```

Esses comandos trabalham principalmente com:

REGISTROS



CRUD representa quatro operações fundamentais sobre dados:

Operação	Significado	SQL
Create	Criar registro	INSERT
Read	Ler registro	SELECT
Update	Atualizar registro	UPDATE
Delete	Excluir registro	DELETE



No CRUD:

CREATE

significa criar um **registro**.

No SQL fazemos isso com:

INSERT

Já:

CREATE TABLE

pertence ao DDL e cria uma **estrutura**.



```
CREATE TABLE cliente
```

Cria:

```
A TABELA
```

Enquanto:

```
INSERT INTO cliente ...
```

cria:

```
UM REGISTRO NA TABELA
```

Essa diferença deve permanecer clara durante toda a aula.



Inserindo dados com INSERT

Estrutura básica:

```
INSERT INTO tabela (coluna1, coluna2)  
VALUES (valor1, valor2);
```

Exemplo:

```
INSERT INTO cliente (nome, email)  
VALUES ('Ana', 'ana@email.com');
```



O que aconteceu?

Executamos:

```
INSERT INTO cliente (nome, email)
VALUES ('Ana', 'ana@email.com');
```

Mas não informamos:

```
id_cliente
ativo
criado_em
```

Por quê?

Porque essas colunas possuem mecanismos automáticos.



Na definição da tabela:

```
id_cliente BIGINT  
GENERATED BY DEFAULT AS IDENTITY
```

O PostgreSQL gera o identificador.

Também definimos:

```
ativo BOOLEAN DEFAULT TRUE
```

e:

```
criado_em TIMESTAMPTZ DEFAULT NOW()
```

Portanto, o banco consegue completar o registro.



Inserimos:

```
INSERT INTO cliente (nome, email)
VALUES ('Ana', 'ana@email.com');
```

O banco poderá armazenar algo equivalente a:

id_cliente	nome	email	ativo	criado_em
1	Ana	ana@email.com	true	horário da inserção

Parte dos valores foi fornecida.

Parte foi gerada pelo banco.



Também podemos fornecer valores explicitamente:

```
INSERT INTO cliente (  
    nome,  
    email,  
    ativo  
)  
VALUES (  
    'Bruno',  
    'bruno@email.com',  
    FALSE  
);
```

Ainda assim:

```
id_cliente → gerado automaticamente  
criado_em → DEFAULT NOW()
```



Inserindo vários registros

Podemos inserir várias linhas em um único comando:

```
INSERT INTO cliente (nome, email)
VALUES
  ('Carlos', 'carlos@email.com'),
  ('Daniela', 'daniela@email.com'),
  ('Eduardo', 'eduardo@email.com');
```



Lembre-se:

```
INSERT  
  ↓  
não ignora  
  ↓  
RESTRIÇÕES
```

O banco continua verificando:

```
PRIMARY KEY  
FOREIGN KEY  
NOT NULL  
UNIQUE  
CHECK
```



Considere:

```
INSERT INTO cliente (email)
VALUES ('novo@email.com');
```

Mas:

```
nome VARCHAR(100) NOT NULL
```

Logo:

```
nome ausente
  ↓
NOT NULL
  ↓
operação rejeitada
```



Já existe:

```
ana@email.com
```

Tentamos:

```
INSERT INTO cliente (nome, email)
VALUES ('Outra Ana', 'ana@email.com');
```

Como email possui:

```
UNIQUE
```

o banco deve rejeitar a duplicidade.



Suponha que exista:

```
id_cliente = 1
```

Podemos inserir:

```
INSERT INTO pedido (valor, id_cliente)
VALUES (250.00, 1);
```

Não precisamos informar:

```
id_pedido
data
```

porque:

```
id_pedido → IDENTITY
data      → DEFAULT NOW()
```



```
INSERT INTO pedido (valor, id_cliente)
VALUES (-100.00, 1);
```

Existe:

```
CHECK (valor >= 0)
```

Portanto:

```
-100
  ↓
viola CHECK
  ↓
INSERT rejeitado
```



```
INSERT INTO pedido (valor, id_cliente)
VALUES (500.00, 999);
```

Se o cliente 999 não existir:

```
PEDIDO.id_cliente = 999
    ↓
FOREIGN KEY
    ↓
CLIENTE.id_cliente = ???
```

A inserção deve ser rejeitada.



Agora conseguimos perceber por que definimos restrições.

MODELO FÍSICO



RESTRIÇÕES



INSERT



SGBD VALIDA



DADO ACEITO OU REJEITADO



No Supabase/PostgreSQL podemos solicitar que o banco retorne o registro inserido:

```
INSERT INTO cliente (nome, email)
VALUES ('Fernanda', 'fernanda@email.com')
RETURNING *;
```

Isso permite visualizar imediatamente o resultado.



```
INSERT INTO cliente (nome, email)
VALUES ('Gabriel', 'gabriel@email.com')
RETURNING id_cliente;
```

Isso é especialmente útil quando o identificador foi gerado por:

```
GENERATED BY DEFAULT AS IDENTITY
```



Consultando dados com SELECT

Depois de inserir registros, precisamos consultá-los.

Comando:

```
SELECT
```

Consulta mais simples:

```
SELECT *  
FROM cliente;
```



```
SELECT *  
FROM cliente;
```

Podemos ler:

Selecione todas as colunas de todos os registros da tabela cliente.

O símbolo:

*

representa todas as colunas.



```
SELECT nome, email  
FROM cliente;
```

Agora não queremos toda a estrutura do registro.

Queremos apenas:

```
nome  
email
```

Isso é uma **projeção** dos dados.



Tabela:

id_cliente	nome	email	ativo
1	Ana	ana@email.com	true
2	Bruno	bruno@email.com	false

Consulta:

```
SELECT nome, email  
FROM cliente;
```

Resultado:

nome	email
Ana	ana@email.com
Bruno	bruno@email.com



Nem sempre queremos todos os registros.

```
SELECT *  
FROM cliente  
WHERE id_cliente = 1;
```

WHERE define uma condição.

```
TODOS OS CLIENTES  
    ↓  
WHERE  
    ↓  
somente os que satisfazem a condição
```



Podemos utilizar:

```
=      igual
<>     diferente
>      maior
<      menor
>=     maior ou igual
<=     menor ou igual
```

Exemplo:

```
SELECT *
FROM pedido
WHERE valor >= 500;
```



```
SELECT *  
FROM pedido  
WHERE valor >= 100  
      AND valor <= 500;
```

Também podemos utilizar:

```
OR
```

Exemplo:

```
SELECT *  
FROM cliente  
WHERE nome = 'Ana'  
      OR nome = 'Bruno';
```



AND↓

todas as condições precisam ser verdadeiras

OR↓

pelo menos uma condição precisa ser verdadeira

Exemplo:

```
SELECT *  
FROM cliente  
WHERE ativo = TRUE  
      AND id_cliente >= 2;
```



Não utilizamos:

```
telefone = NULL
```

Utilizamos:

```
telefone IS NULL
```

ou:

```
telefone IS NOT NULL
```

Exemplo:

```
SELECT *  
FROM cliente  
WHERE telefone IS NULL;
```



```
SELECT *  
FROM pedido  
ORDER BY valor ASC;
```

ASC:

crescente

Para ordem decrescente:

```
SELECT *  
FROM pedido  
ORDER BY valor DESC;
```



No PostgreSQL/Supabase:

```
SELECT *  
FROM pedido  
LIMIT 5;
```

Isso retorna no máximo cinco registros.

Também podemos combinar:

```
SELECT *  
FROM pedido  
ORDER BY valor DESC  
LIMIT 5;
```



```
SELECT
    id_pedido,
    valor
FROM pedido
WHERE valor >= 100
ORDER BY valor DESC
LIMIT 5;
```

Leia na ordem lógica:

```
FROM pedido
↓
WHERE valor >= 100
↓
SELECT id_pedido, valor
↓
ORDER BY valor DESC
↓
LIMIT 5
```



Podemos pesquisar padrões com:

LIKE

Exemplo:

```
SELECT *  
FROM cliente  
WHERE nome LIKE 'Ana%';
```

O % representa zero ou mais caracteres.



ILIKE no PostgreSQL

PostgreSQL também fornece:

ILIKE

Exemplo:

```
SELECT *  
FROM cliente  
WHERE nome ILIKE 'ana%';
```

Isso é útil para buscas sem diferenciação entre maiúsculas e minúsculas de acordo com as regras aplicáveis pelo PostgreSQL.



Atualizando registros

Estrutura:

```
UPDATE tabela  
SET coluna = valor  
WHERE condicao;
```

Exemplo:

```
UPDATE cliente  
SET ativo = FALSE  
WHERE id_cliente = 1;
```



```
UPDATE cliente  
SET ativo = FALSE  
WHERE id_cliente = 1;
```

Podemos ler:

```
cliente  
↓  
localize id_cliente = 1  
↓  
altere ativo  
↓  
FALSE
```



```
UPDATE cliente
SET
    nome = 'Ana Silva',
    email = 'ana.silva@email.com'
WHERE id_cliente = 1;
```

Um único UPDATE pode modificar várias colunas do mesmo conjunto de registros.



O perigo do UPDATE sem WHERE

Considere:

```
UPDATE cliente  
SET ativo = FALSE;
```

Sem:

```
WHERE ...
```

o comando pode alterar:

```
TODOS OS REGISTROS
```

Portanto:

Antes de executar um UPDATE, verifique cuidadosamente a condição do WHERE.



Antes:

```
SELECT *  
FROM cliente  
WHERE id_cliente = 1;
```

Confira o resultado.

Depois:

```
UPDATE cliente  
SET ativo = FALSE  
WHERE id_cliente = 1;
```

A ideia é:

```
SELECT  
↓  
verifico quem será afetado  
↓  
UPDATE
```



No PostgreSQL:

```
UPDATE cliente  
SET ativo = FALSE  
WHERE id_cliente = 1  
RETURNING *;
```

Assim podemos visualizar os registros modificados.



Considere:

```
UPDATE pedido  
SET valor = -500  
WHERE id_pedido = 1;
```

Se existe:

```
CHECK (valor >= 0)
```

o banco deve rejeitar a alteração.

As restrições não protegem apenas o INSERT.

Também protegem alterações posteriores.



```
UPDATE pedido  
SET id_cliente = 999  
WHERE id_pedido = 1;
```

Se o cliente 999 não existir:

```
UPDATE  
↓  
FOREIGN KEY  
↓  
referência inválida  
↓  
operação rejeitada
```



Excluindo registros

Estrutura:

```
DELETE FROM tabela  
WHERE condicao;
```

Exemplo:

```
DELETE FROM cliente  
WHERE id_cliente = 5;
```



```
DELETE FROM cliente  
WHERE id_cliente = 5;
```

Podemos ler:

Remova da tabela cliente o registro cujo id_cliente seja igual a 5.



Considere:

```
DELETE FROM cliente;
```

Sem WHERE, todos os registros selecionáveis pela operação podem ser removidos.

Portanto:

```
DELETE + WHERE
```

deve receber a mesma atenção que:

```
UPDATE + WHERE
```



Primeiro:

```
SELECT *  
FROM cliente  
WHERE id_cliente = 5;
```

Depois:

```
DELETE FROM cliente  
WHERE id_cliente = 5;
```

Fluxo:

```
SELECT  
↓  
confirmo o registro  
↓  
DELETE
```



```
DELETE FROM cliente  
WHERE id_cliente = 5  
RETURNING *;
```

O PostgreSQL retorna os registros removidos pela instrução.

Mas existe uma FOREIGN KEY



Lembre-se:

```
CLIENTE 1 ----- N PEDIDO
```

E definimos:

```
ON DELETE RESTRICT
```

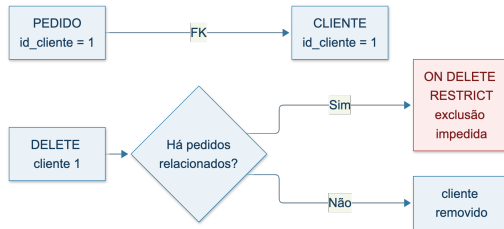
O que acontece se tentarmos excluir um cliente que possui pedidos?

Integridade referencial durante DELETE



```
DELETE FROM cliente  
WHERE id_cliente = 1;
```

Se existem pedidos:



Com:

```
ON DELETE RESTRICT
```

a exclusão será impedida.



Porque remover o cliente produziria:

```
PEDIDO  
id_cliente = 1  
  ↓  
CLIENTE  
não existe
```

Isso criaria uma referência inválida.

Portanto:

```
DELETE  
  ↓  
FOREIGN KEY  
  ↓  
integridade referencial
```



Agora conseguimos realizar:



Essas operações formam a base da manipulação de dados.



O CRUD não opera livremente.

```
INSERT  
UPDATE  
DELETE  
  ↓  
passam pelas  
  ↓  
RESTRIÇÕES DO BANCO
```

Exemplos:

```
NOT NULL  
UNIQUE  
CHECK  
PRIMARY KEY  
FOREIGN KEY
```



MODELO RELACIONAL



MODELO FÍSICO



CREATE TABLE



RESTRIÇÕES



DADOS



CRUD

O que definimos durante a modelagem influencia diretamente quais operações serão aceitas pelo banco.



Considere:

CATEGORIA

id_categoria PK

nome

PRODUTO

id_produto PK

nome

preco

estoque

id_categoria FK

Relacionamento:

CATEGORIA 1 ----- N PRODUTO



No Supabase, crie:

```
CREATE TABLE categoria (  
  id_categoria BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL UNIQUE  
);
```

Depois crie produto com:

- id_produto automático;
- nome obrigatório;
- preco maior ou igual a zero;
- estoque iniciando em zero;
- id_categoria como chave estrangeira.



Insira pelo menos três categorias:

```
Informática  
Escritório  
Acessórios
```

Depois insira pelo menos cinco produtos.

Utilize:

```
INSERT
```

e observe os IDs gerados automaticamente.



Crie consultas para:

- 1 listar todos os produtos;
- 2 mostrar apenas nome e preço;
- 3 localizar produtos com preço maior que 100;
- 4 localizar produtos com estoque igual a zero;
- 5 ordenar produtos do maior para o menor preço;
- 6 retornar apenas os três primeiros resultados.



Realize:

- ① alteração do preço de um produto;
- ② alteração do estoque;
- ③ alteração de nome;
- ④ tentativa de definir preço negativo.

Antes de cada UPDATE:

```
SELECT ...
```

Depois utilize:

```
RETURNING *;
```



Exercício guiado — DELETE

- 1 selecione um produto;
- 2 confirme seu identificador;
- 3 exclua-o;
- 4 utilize `RETURNING *`;
- 5 tente excluir uma categoria que ainda possua produtos.

Analise:

Qual restrição interfere na última operação?



Para cada comando executado, registre:

Operação	SQL	Resultado	Restrição envolvida
Inserir produto válido	INSERT	Aceito	—
Inserir preço negativo	INSERT	Rejeitado	CHECK
Alterar FK para categoria inexistente	UPDATE	Rejeitado	FOREIGN KEY
Excluir categoria utilizada	DELETE	Rejeitado	FOREIGN KEY

Complete com novos experimentos.



Quando o Supabase/PostgreSQL rejeitar um comando:

NÃO apenas corrija

Investigue:

1. Qual operação foi realizada?
2. Qual restrição foi violada?
3. Qual regra do modelo essa restrição representa?
4. Qual alteração tornaria a operação válida?



Durante esta aula utilizaremos o **SQL Editor do Supabase** para estudar diretamente o comportamento do PostgreSQL.

Posteriormente, quando o banco for acessado por uma aplicação, também precisaremos discutir:

```
AUTENTICAÇÃO  
+  
AUTORIZAÇÃO  
+  
ROW LEVEL SECURITY (RLS)
```

Esses mecanismos tratam de **quem pode acessar quais dados**.



Não confunda:

INTEGRIDADE↓

o dado é válido?

com:

SEGURANÇA / RLS↓

quem pode ler ou modificar o dado?

Nesta etapa, nosso foco principal continua sendo:

SQL

+

CRUD

+

INTEGRIDADE



Estrutura

```
CREATE TABLE  
ALTER TABLE  
DROP TABLE
```

Dados

```
INSERT  
SELECT  
UPDATE  
DELETE
```

Apoio importante do PostgreSQL

```
RETURNING
```



DDL↓

DEFINE A ESTRUTURA

CREATE TABLE

ALTER TABLE

DROP TABLE

↓

DML / CONSULTA↓

TRABALHA COM OS DADOS

INSERT

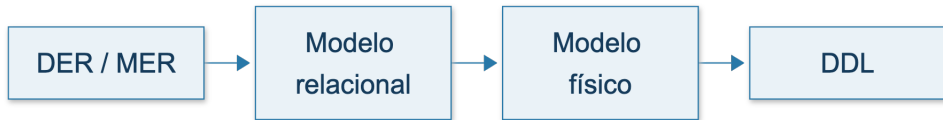
SELECT

UPDATE

DELETE



Da modelagem à estrutura



Da estrutura aos dados





Até aqui, consultamos principalmente:

```
UMA TABELA POR VEZ
```

Mas nosso banco possui relacionamentos:

```
CLIENTE 1 ----- N PEDIDO
```

A próxima pergunta será:

Como consultar dados que estão distribuídos entre tabelas relacionadas?

Isso nos levará a:

```
JOIN
```



Consultas relacionais

Vamos evoluir de:

```
SELECT *  
FROM pedido;
```

para consultas como:

```
SELECT  
    p.id_pedido,  
    c.nome,  
    p.valor  
FROM pedido AS p  
INNER JOIN cliente AS c  
    ON p.id_cliente = c.id_cliente;
```

Agora o relacionamento definido no modelo também passará a ser utilizado nas **consultas**.